<!-- Extracted from https://oracc.museum.upenn.edu/doc/help/editinginatf/primer/structuretutorial/index.html on 2026-10-16 by a stdlib HTML->Markdown converter; see raw .html for the original. -->

ATF Structure Tutorial

This document gives a tutorial on how to type the block structure features of texts in ATF, the ASCII Transliteration Format.

Introduction

We provide a simple introduction to typing ATF texts, describing the more common features first and filling in the details later. Before explaining any specifics, here is a simple typical example of an ATF text:

```
&P555555 = Some Publication 32
@obverse
1. 1(diš) udu ba-ug₇
$ reverse blank
```

This example illustrates the four of the most common types of lines in an ATF text:

&-lines
: Every ATF text must start with an `&-lines` ("and-lines") which normally gives the CDLI P-identifier and should also have a human-readable name following it after an '=' sign.

@-lines
: Divisions in the text are specified using lines that start with an `@` sign ("at-lines"). These are used to indicate object types, surfaces, divisions and columns.

$-lines
: Descriptive asides concerning the preservation or state of the text are given using `$-lines` ("dollar lines"). These look like ordinary sequences of words but they may be subject to strict rules.

Text lines
: Lines beginning with non-spaces followed by a period followed by one or more spaces are lines of text. The rules for transliteration are given in the ATF Grapheme tutorial.

Line-types

#-lines

The other quite common type of line in an ATF file begins with the hash sign (`#`). There are two kinds of #-line: protocols and comments.

Protocols

Protocols are statements which are interpreted or stored by the ATF processor but are not part of the text edition proper. Protocols are all named and may trigger special processing within the ATF processor.

The details of protocols are beyond the scope of this tutorial; for now, it is enough to know that they look like this:

```
&P123321 = Some Akkadian Text
#atf: lang akk-x-oldbab
1. i-na AN{+e}
#note: This is a contrived note.
```

Most protocols are a single line and do not require a blank line after them to separate them from a following protocol (the one exception is `#note:`).

More information on protocols, what they are, where they are allowed and the rules about ordering of protocols is available in the protocols manual.

Comments

Comments are asides which are not part of the text edition or the annotation; they are useful for keeping odd bits of information in the file without it getting in the way of the text edition or annotation.

Comments are indicated in ATF by one or more lines beginning with the hash character (`#`).

Comments look like protocols in that they begin with a hash-sign, but they may not begin with the sequence hash-name-colon. Comments may be included within text transliterations but not before the first text in a file. Comments must always follow any protocols which occur adjacent to them.

A sequence of lines beginning with hash-signs is a multi-line comment. To separate multiple comments to the same line use a blank line in the ATF file.

```
1. a
#a simple comment

2. a
#a longer comment which somewhat artificially extends
#over multiple lines

3. a
#one comment to line 3.

#another comment to line 3.

4. a
#Comments look a bit like protocols but there is no chance of
#confusion: the ATF processor's scanning rules take care of that.

5. a
#lem: a[water]
#note: If you want a comment to appear in the displayed text-edition
#use the '#note:' protocol instead.

#and note that any comment must follow any other protocol, including
#'#note:'.
```

You can include note marks in the transliteration and after an `#note:` by putting the note mark between caret signs (e.g., `^1^`). You can also specify that a note corresponds to the label of a text line (or a range) by using the `@notelabel{...}` notation, e.g., `#note: @notelabel{i 1} A note to column 1 line 1.`.

&-lines

&-lines are used to introduce a new text and consist of two parts: the ID and the name.

For transliterations of exemplars, the ID is a 'P' followed by six digits, e.g., P123456. This ID is assigned by CDLI and is the reference ID of the object in the main CDLI catalog; to get IDs for objects not in the CDLI catalog send an e-mail to cdli@cdli.ucla.edu.

The name of the text should be identical with the 'Designation' field in the CDLI main catalog; the ATF processor detects mismatches and reports the correct name. This mechanism is designed to provide a check that the P-number in the ID actually references the text the transliterator intends.

In ATF the two parts of an &-line are separated by space-equals-space, like this:

```
&P000001 = ATU 3, pl. 011, W 6435,a
```

@-lines

@-lines are used for structural tags. Several kinds of structure may be indicated using this mechanism: physical structure, e.g., objects, surfaces; manuscript structure, i.e., columns; and document structure, e.g., divisions and colophons. For clarity, we describe here only the structural features which are permitted in object transliterations, i.e., texts with an ID beginning with `P`. Documentation of structural conventions for composite texts is given in the composites manual.

Objects

The kind of object on which the inscription being transliterated is written is designated using one of the following tags:

@tablet
: The default, and therefore optional; object is a tablet.

@envelope
: Tablets and envelopes with the same P number can be transliterated separately using this tag.

@prism
: Object is a prism.

@bulla
: Object is a bulla.

@fragment
: Object is a fragment, with a fragment name (e.g., a letter) following the tag; may be used more than once to transliterate multiple fragments of an object, e.g.:

```
&P212121 = Some Fragmentary Object
@fragment a
1. a
@fragment b
1. a
```

`@object`
: The generic object tag which must be followed by the type of the object, e.g. `@object Stone wig`.

Seals

A transliteration of the text inscribed on a physical seal object should be handled using the `@object` tag:

```
&P333444 = Some Seal
@object seal
1. da-da
2. dumu du-du
```

Surfaces

Surfaces are principally the physical surfaces:

`@obverse`, **`@reverse`**
: Obverse and reverse.

`@left`, **`@right`**, **`@top`**, **`@bottom`**
: Specifiable edges, left right, top and bottom (as seen when looking at obverse of tablet).

`@face`
: Conventional designation for surfaces of a prism; must be followed by single lowercase letter indicating the face, e.g.:

```
&P123321 = Some Prism
@prism
@face a
1. a
@face b
1. e
```

`@surface`
: Generic surface tag which must be followed by name of surface, e.g.: `@surface shoulder`; `@surface side a`.

`@edge`
: Generic edge tag; may be followed by single lowercase letter to name the edge similarly to `@face`.

Sealings

A transliteration of a sealing should be handled using the `@seal` tag included like a surface after the transliteration of the object on which the sealing occurs:

```
&P343434 = Some Sealed Tablet
1. a
$ seal 1

@seal 1
1. du-du
```

The use of `$ seal` anticipates the discussion of $-lines below; this mechanism can be used to indicate which sealings occur where on an object.

Columns

Columns are indicated with the `@column` tag, which may be omitted for single-column texts. Column numbers must be given in arabic numerals:

```
&P545454 = Some Columnar Text
@column 1
1. a
@column 2
1. e
```

Status

The status of some of the features indicated with @-lines can be indicated in a manner similar to that of graphemes; the notation is intended to be natural and to follow Assyriological conventions:

```
@obverse?
```

Meaning: status of obverse/reverse uncertain

```
@reverse!*
```

Meaning: collated; reverse correct despite designation in publication

Primes can be used where this makes sense:

```
@face a'

@column 3'
```

Headings

Transliterations and composites can both contain headings, which take the form `@h<DIGIT>`, where DIGIT is the outline-level of the heading, normally 1, 2 or 3.

Milestones

For technical reasons it is impossible to interweave physical structure (of the kind described above for transliterated objects) and document structure (e.g., paragraph divisions). This limitation is resolved by recourse to milestones.

Divisions

Documentary divisions in a transliterated object are given using the `@m` tag, with the milestone type given after an equals sign and the division type following; an optional division name or number may follow the division type:

```
@m=division paragraph 1

@m=division colophon
```

Discourse

Simple support for discourse elements in administrative and scholarly texts is provided using shorthands which are also implemented as milestones. These shorthands are:

- `@catchline`
- `@colophon`
- `@date`
- `@signatures` and `@signature`
- `@summary`
- `@witnesses`

These milestones must be specified between lines. If you need to mark a milestone in the middle of a line then you can split the line into two (labeled, e.g., a and b) at the milestone.

```
&P787878 = Some Administrative Text
1. 1(diš) udu
2. da-da
3. šu ba-ti
@date
4. u₄ 1-kam
@left
@summary
1. 1(diš) udu
```

```
&P908908 = A Scholarly Text
@colophon
1a. UNUG{ki}
@date
1b. {iti}AB U₄ 1-KAM₂
2. MU 1.39@v-KAM₂ {m}an-ti-ʾi-ku-su LUGAL
```

$-lines

$-lines are used to indicate information about the state of the text or object, or to describe features on the object which are not part of the transliteration proper. They come in two flavours: strict and loose.

Strict $-lines are subject to the restrictions in the table below; strict $-lines can be interpreted in their entirety by the ATF processor and the interpreted information can then be used by other programs. Strict $-lines are the best practice.

Loose $-lines are indicated by putting parentheses around the contents of the $-line. This is a facility provided to enable annotation of features which are not covered by the strict $-line specification. If the ATF processor detects that a loose $-line actually meets the criteria defined for strict $-lines it gives an advisory notice that the parentheses should be removed.

$-lines and comments are two quite different facilities, but experience has shown that transliterators can confuse the two. Comments are for information which does not belong in the transliteration and description of the text; comments are not displayed when the text is formatted for display or print. $-lines are for information which is integral to an understanding of the textual data; $-lines are included when the text is displayed or printed.

Seal

A particular use of $-lines is to indicate that a seal is used on an object; the form is:

```
$ seal <N>
```

Where `N` is a number indicating which seal is used; if a transliteration of the seal is also given using the `@seal` heading, the number following `$ seal` should correspond to the number following `@seal`. See the example above.

State

Most $-lines are used to give information about the state of the object being transliterated. The conventions for this can be summarized as follows:

Summary of Strict $-line Conventions for States
| Qualification | Extent1 | Scope | State

| 1The extent `N` may be a number such as 1 or 5; a `RANGE` gives two numbers separated by a hyphen, e.g., 3-5.

| 2`OBJECT` is any object specifier as described above, e.g., tablet, object etc.

| 3`SURFACE` is any surface specifier as described above, e.g., obverse, left etc.

| at least

at most

about | n

several

some

NUMBER

RANGE

rest of

start of

beginning of

middle of

end of

| OBJECT2

SURFACE3

column

columns

line

lines

case

cases

surface | blank

broken

effaced

illegible

missing

traces

Rulings

$-lines are also used to indicate noteworthy rulings on the tablet; ordinary case- or line-ruling should not be indicated with a $-line, but where a scribe has used a ruling to give additional information about the document structure this should be noted as:

```
(single | double | triple)   ruling
```

Examples

Strict $-lines look like this:

```
$ 3 lines blank
$ rest of obverse missing
```

A loose $-line looks like this:

```
$ (head of statue broken)
```

A ruling $-line looks like this:

```
$ double ruling
```

Images

Inline images can be specified using the form:

```
$ (image N = <text>)
```

Where N is an image number consisting of digits followed by optional lowercase letters from a to z, and <text> is free text, giving a label for the image (which is copied through to the XHTML 'alt' attribute on the <img> tag).

```
$ (image 1 = numbered diagram of triangle)
```

At present, the implementation only works for XHTML which is produced within a project. The ATF processor constructs a file name consisting of the text ID and the image's N value, joined by an at sign (e.g., `P123456@1`). The XHTML producer then emits an `<img>` tag with the `src` attribute set to `/<PROJECT>/<FILENAME>.png`.

Thus, in the present implementation, there must exist an appropriately named file in the PNG graphics format residing in the project's `images` directory. The implementation is expected to support a more sophisticated locator mechanism in the future.

Text Lines

Lines of transliterated text begin with a sequence of non-space characters followed by a period and a space (these are typically numbers, but that is not a requirement):

```
1.   a
a+1. e
2'.  i
```

In ATF, lines containing only spaces are ignored; lines beginning with a space are continuation lines and the newline and leading spaces are dropped by the ATF processor:

```
1. a a a a
   a a a
```

The content of lines is defined principally by the Grapheme Description Language, but there are some line-related ATF features which are not necessary for many users and which are dealt with in the advanced documentation.
18 Dec 2019 osc at oracc dot org

Steve Tinney

Steve Tinney, 'ATF Structure Tutorial', *Oracc: The Open Richly Annotated Cuneiform Corpus*, Oracc, 2019 [http://oracc.museum.upenn.edu/doc/help/editinginatf/primer/structuretutorial/]

Back to top ^^

Released under a Creative Commons Attribution Share-Alike license 3.0, 2014.
[http://www.facebook.com/opencuneiform] [http://oracc.blogspot.com]
[http://www.twitter.com/oracctivity]

Oracc uses cookies only to collect Google Analytics data. Read more here; see the stats here [http://www.seethestats.com/site/oracc.museum.upenn.edu]; opt out here.